AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

EXERCISE 20 · UNIT 3

Parallel reduction on a GPU

Trivial on a CPU, instructive on a GPU, because there is no synchronisation between thread blocks.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

EXERCISE	UNIT
ex20-cuda-reduction	3 — GPU and CUDA
LANGUAGE	MARKS
CUDA C++	10
SUBMIT AS	OFFERING
<ROLLNO>.cu	Odd Semester 2026-27

The problem

Sum an array on the device, and find its maximum. Both take host pointers and return a single value.

The one hard fact

DEFINITION · THERE IS NO SYNCHRONISATION BETWEEN THREAD BLOCKS

A block cannot wait for other blocks. The runtime may schedule them in any order, on any SM, and two given blocks may never be resident at the same time.

So **no single kernel launch can combine contributions from the whole grid**.

The standard answer is a **two-stage reduction**:

```
stage 1: each block reduces its own slice and writes ONE partial result
stage 2: a second launch, or a small host pass, reduces the partials
```

The kernel launch boundary *is* the global barrier you cannot get any other way.

The tree, inside a block

```
__shared__ float sdata[BLOCK];
sdata[tid] = my_value;
__syncthreads();

for (unsigned s = blockDim.x / 2; s > 0; s >>= 1) {
    if (tid < s) sdata[tid] += sdata[tid + s];
    __syncthreads();               /* EVERY thread reaches this */
}
if (tid == 0) out[blockIdx.x] = sdata[0];
```

PITFALL · WHY THE ACTIVE THREADS ARE THE LOW ONES

Halving the stride and keeping the active threads **contiguous** (`tid < s`) means whole warps retire together. The older textbook form, `if (tid % (2*s) == 0)` , leaves every warp half active, which is warp divergence on every single step of the tree.

PITFALL · __SYNCTHREADS() MUST NEVER SIT INSIDE A DIVERGENT BRANCH

Every thread in the block must reach it. Putting it inside the `if (tid < s)` hangs the block. Note where it sits in the loop above: inside the loop, outside the `if` .

The identity matters

`n` is not a multiple of the block size, so some threads have no element. They must contribute the **identity of the operator**:

OPERATOR	IDENTITY
sum	`0.0f`
maximum	`-INFINITY`

PITFALL · CONTRIBUTING 0 TO A MAXIMUM

It works on every test with a positive number in it, and silently returns 0 for an all-negative array. Two of the tests are all-negative for exactly this reason.

Precision

`gpu_sum` returns a `double` even though the input is `float`. Accumulate the per-block partials in `double` on the host: the tree reduction inside each block already keeps the error far lower than a serial `float` sum would, and throwing that away in the final combination would be a waste.

Building and testing

```
./selfcheck.sh            # compiles and runs the public tests through srun
```

ON ASAICOMPUTE · YOU NEED A GPU

`selfcheck.sh` submits through `srun` automatically when SLURM is available. Compiling on the head node is fine; running is not.

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Warnings as errors
Uses a shared-memory tree reduction	2	`__shared__` and `__syncthreads()` present; no per-element `atomicAdd`
Public tests	3	Sums, maxima, empty input, all-negative input
Hidden tests	4	Block boundaries, 4M elements, repeatability, extreme positions

A per-element `atomicAdd` on the input is rejected. It gives the right answer, and it is not what this exercise teaches.

Submitting

```
AIE23001.cu
```